Nüber (Uber-clone) – AWS Architecture Design Document

Abstract

This report outlines the design of *Nüber*, a cloud-based ride-hailing platform modeled on Uber. It aims to provide a globally scalable, secure, and extensible service that connects riders, drivers, and administrators. The design leverages **Amazon Web Services (AWS)** as its backbone, supported by trusted third parties for payments, mapping, crash reporting, and music. Key considerations include global availability, privacy and data residency compliance, defense-in-depth security, and readiness for future service extensions (e.g., food or courier delivery). The resulting architecture balances performance, compliance, and cost-efficiency.

System Overview

Nüber supports three user roles:

- **Administrators** – compliance, onboarding, and system settings.
- **Drivers** – register, upload documents, and accept trips.
- **Riders** – request rides, track progress, pay securely, and optionally share music.

Core features include authentication (Cognito), document storage (S3), payments (Stripe), routing (Google Maps), music preferences (Spotify), analytics (Kinesis/Athena), and crash reporting (Sentry).

The system is deployed in Sydney, Ireland, and Virginia to provide coverage for APAC, Europe/UK, and North America. China and some Middle Eastern countries are excluded due to regulatory complexity and data residency laws.

Architecture

Nüber adopts a layered, cloud-native design on AWS, balancing scalability, security, and extensibility. The system is organized into six tiers: edge access, identity, application services, data, analytics/observability, and external integrations.

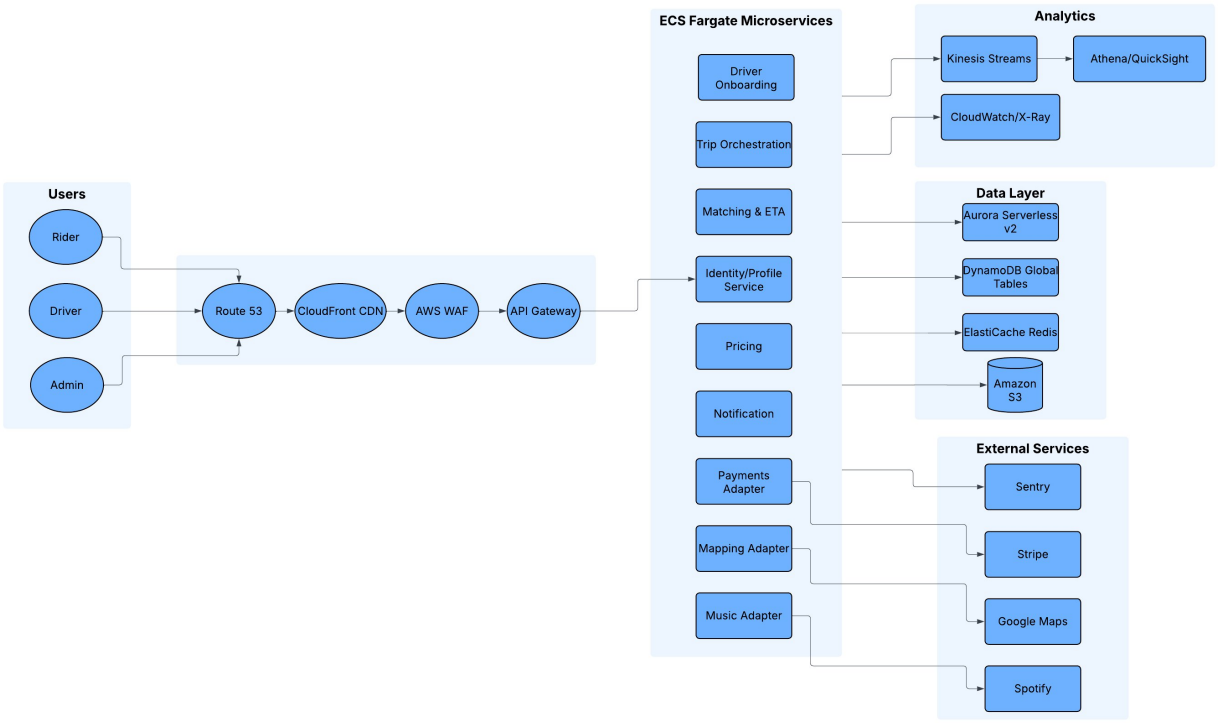


Figure 1: High Level Architecture Diagram

Key components:

- **Edge & Access:** Route 53 (DNS), CloudFront (CDN), and WAF secure and optimize incoming traffic. API Gateway manages REST and WebSocket entry points.
- **Identity:** Amazon Cognito provides authentication, MFA, and federated logins, issuing JWT tokens with role-based claims.
- **Application Services (ECS Fargate):** Microservices handle Identity/Profile, Driver Onboarding, Matching & ETA, Pricing, Trip Orchestration, Payments, Mapping, Music, and Notifications.
- **Data Layer:** Aurora Serverless v2 (PostgreSQL) for transactions and PII; DynamoDB Global Tables for high-throughput geospatial data; ElastiCache (Redis) for hot lookups; S3 for documents and receipts.
- **Analytics & Observability:** Kinesis → Firehose → S3 → Athena/QuickSight for analytics. CloudWatch and X-Ray monitor operations; Sentry captures crashes.
- **External Services:** Stripe (payments), Google Maps (routes), Spotify (music), and Sentry (crash reporting).

Multi-Region Deployment:

- Regions: ap-southeast-2 (Sydney), eu-west-1 (Ireland), us-east-1 (Virginia).
- 3 key regions balance performance with data residency.

- Aurora Global Database and DynamoDB Global Tables replicate data where needed.
- Route 53 latency-based routing directs users to the nearest region, ensuring low-latency access while keeping PII in-region.

Data Design

Nüber applies a polyglot persistence model to balance consistency, performance, and compliance:

- **Aurora Serverless v2 (Postgres):** Relational store for users, trips, payments, and PII (encrypted with KMS, IAM-scoped).
- **DynamoDB Global Tables:** High-throughput geospatial and surge data, replicated across regions for low-latency matching.
- **ElastiCache (Redis):** Caches hot queries such as nearby drivers and active sessions.
- **Amazon S3:** Stores driver documents and receipts with Object Lock and lifecycle policies.

Residency tags ensure PII remains in-region, while anonymized aggregates feed analytics pipelines. This model supports compliance (GDPR) and scales with workload demands.

Scalability

Nüber scales both by region and by workload:

- **Geographic scaling** - Sydney, Ireland, and Virginia deployments with latency-based routing.
- **Elastic services** - ECS Fargate, Aurora Serverless v2, and DynamoDB On-Demand scale automatically by demand.
- **Time-of-day scaling** - commuter peaks, event surges (e.g., New Year's Eve), and low overnight traffic are absorbed automatically.
- **Traffic spikes** - EventBridge/SQS decouple workloads, Kinesis buffers analytics, WAF and API Gateway throttling protect from abuse.
- **Resilience** - Aurora/DynamoDB replication and S3/Aurora PITR ensure continuity.

Security & Privacy

Nüber applies defense-in-depth across all layers:

- **Network:** Private VPC subnets, least-privilege security groups, VPC endpoints.
- **Identity:** Cognito with MFA and JWT role claims (admin, driver, customer).

- **Secrets & Encryption:** Secrets Manager with automatic rotation; all data encrypted at rest with KMS and in transit with TLS 1.2+.
- **Application Protection:** WAF filters attacks, Shield defends against DDoS, API Gateway enforces throttling.

If hacked: a compromised web server would expose only transient tokens. PII, payment details, and documents remain encrypted in Aurora, S3, and Stripe. Separation of data stores limits breach impact.
Right-to-be-Forgotten (GDPR Article 17): Step Functions orchestrate deletion across Cognito, Aurora, DynamoDB, and S3, with hooks to revoke data in Stripe, Spotify, and Sentry. Anonymized analytics are retained, and confirmation is sent to the user.

Third-Party Integrations

Nüber relies on trusted third parties to extend functionality:

- **Stripe:** Processes all payments using tokenized identifiers; Nüber never stores card data. REST + Webhooks, tokenized payments only
- **Google Maps Platform:** REST API for routing/ETA (coordinates are shared).
- **Spotify:** OAuth 2.0 for playlists (metadata, not history).
- **Sentry:** SDK crash reporting (stack traces, anonymized IDs).

Each integration is isolated in its own microservice, secured with Secrets Manager, and included in the purge workflow to ensure GDPR compliance.

Extensibility

Extensibility
Nüber is built for growth beyond ride-hailing, using modular microservices and an event-driven backbone. Core services—**Identity (Cognito), Payments (Stripe), and Notifications (SNS/SES/APNs/FCM)**—are shared across domains, ensuring users maintain one account and one wallet.

New bounded contexts can be added with minimal disruption:

- **Eats (Food Delivery):** adds Order, Menu, and Merchant aggregates.
- **Courier:** introduces CourierJob, Sender, and Receiver.

Both plug into **EventBridge**, subscribing to events like UserRegistered or PaymentCaptured. This design reduces duplication, improves agility, and ensures a seamless cross-service user experience.

Justification

Cost Considerations
At launch, Nüber prioritizes flexibility with serverless and on-demand services:

- ECS Fargate and Aurora Serverless v2 scale automatically, charging only for usage.
- DynamoDB On-Demand absorbs unpredictable location writes.
- CloudFront reduces bandwidth costs by caching content.

As workloads stabilize, costs can be optimized by:

- Using Reserved Instances / Savings Plans for predictable ECS and Aurora usage.
- Switching DynamoDB to provisioned throughput.
- Applying S3 lifecycle policies to move old receipts to Glacier.

This model balances low startup costs with long-term savings.

Service Selections

- **Cognito vs Auth0:** Chose **Cognito** for AWS integration and cost; Auth0 offers more flexibility but is pricier at scale.
- **Aurora vs RDS vs DynamoDB: Aurora** provides strong consistency; **DynamoDB** adds low-latency global replication. RDS lacks Aurora's elasticity.
- **ECS Fargate vs Lambda: Fargate** suits long-running services; Lambda would complicate WebSockets.
- **Stripe vs PayPal/Adyen: Stripe** offers best developer APIs and webhook support; PayPal less flexible, Adyen more complex.
- **Google Maps vs Mapbox: Google Maps** chosen for accuracy and reliability; Mapbox is cheaper but less global.
- **Sentry vs Crashlytics: Sentry** covers both backend and mobile; Crashlytics is mobile-only.
- **Spotify vs Apple Music: Spotify** has wider adoption and richer APIs; Apple Music is regionally limited.